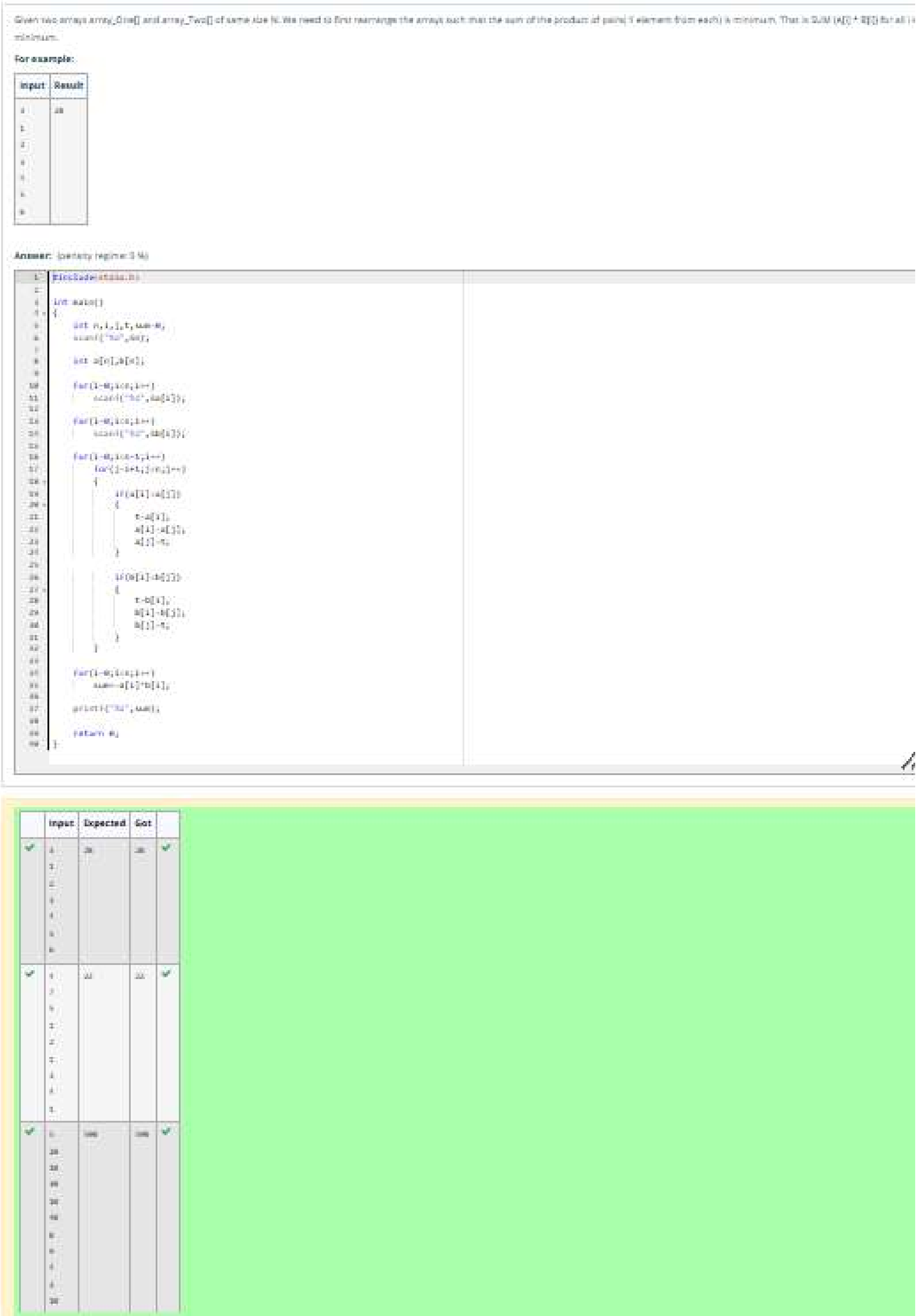


Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i

minimum.

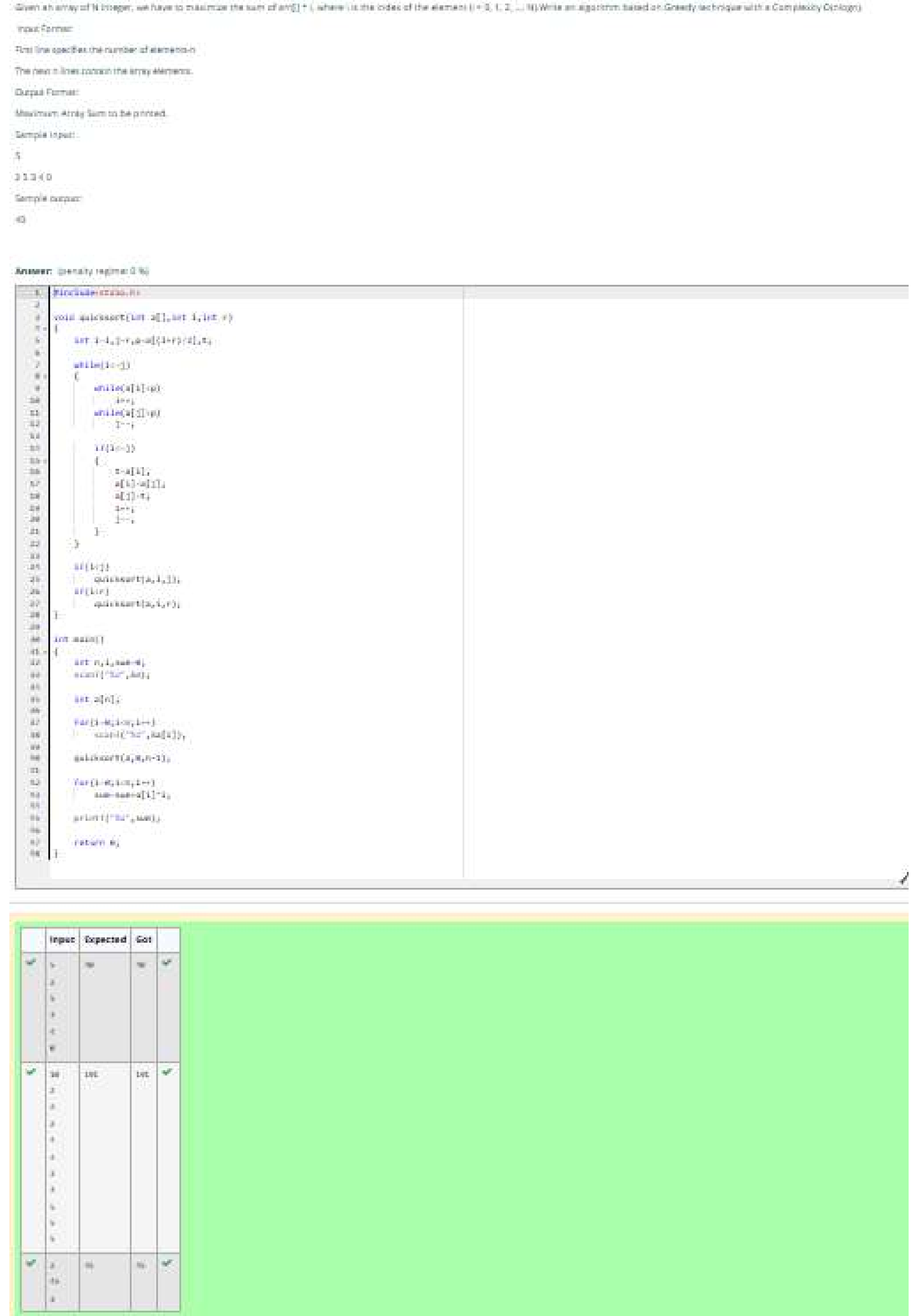
For example:

Input	Result
3 1 2 3 4 5 6	28

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>

int main()
{
    int n,i,j,t,sum=0;
    scanf("%d",&n);

    int a[n],b[n];

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    for(i=0;i<n;i++)
        scanf("%d",&b[i]);

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
        {
            if(a[i]>a[j])
            {
                t=a[i];
                a[i]=a[j];
                a[j]=t;
            }

            if(b[i]<b[j])
            {
                t=b[i];
                b[i]=b[j];
                b[j]=t;
            }
        }

    for(i=0;i<n;i++)
        sum+=a[i]*b[i];

    printf("%d",sum);

    return 0;
}
```

	Input	Expected	Got	
✓	3 1 2 3 4 5 6	28	28	✓
✓	4 7 5 1 2 3 1 4 1	22	22	✓
✓	5 20 10 40 50 40 6 8 4 2 10	[illegible]	[illegible]	✓



Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

Answer: (penalty regime: 0 %)

```
#include<stdio.h>

void quicksort(int a[],int l,int r)
{
    int i=l,j=r,p=a[(l+r)/2],t;

    while(i<=j)
    {
        while(a[i]<p)
            i++;
        while(a[j]>p)
            j--;

        if(i<=j)
        {
            t=a[i];
            a[i]=a[j];
            a[j]=t;
            i++;
            j--;
        }
    }

    if(l<j)
        quicksort(a,l,j);
    if(i<r)
        quicksort(a,i,r);
}

int main()
{
    int n,i,sum=0;
    scanf("%d",&n);

    int a[n];

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    quicksort(a,0,n-1);

    for(i=0;i<n;i++)
        sum=sum+a[i]*i;

    printf("%d",sum);

    return 0;
}
```

	Input	Expected	Got	
✓	5 2 5 3 4 0	40	40	✓
✓	10 2 2 2 4 4 3 3 5 5 5	191	191	✓
✓	2 45 3	45	45	✓

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least $n^i * c$ kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$. But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

3
5 10 7

Sample Output

76

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (x < y)
        return 1;
    if (x > y)
        return -1;
    return 0;
}

int main()
{
    int n;
    scanf("%d", &n);

    int cal[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &cal[i]);

    // Greedy: highest calorie burger first
    qsort(cal, n, sizeof(int), compare);

    long long sum = 0;
    long long power = 1;

    for (int i = 0; i < n; i++)
    {
        sum += (long long)cal[i] * power;
        power *= n;
    }

    printf("%lld", sum);

    return 0;
}
```

	Test	Input	Expected	Got	
✓	Test Case 1	3 1 3 2	18	18	✓
✓	Test Case 2	4 7 4 9 6	389	389	✓
✓	Test Case 3	3 5 10 7	76	76	✓

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i
be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

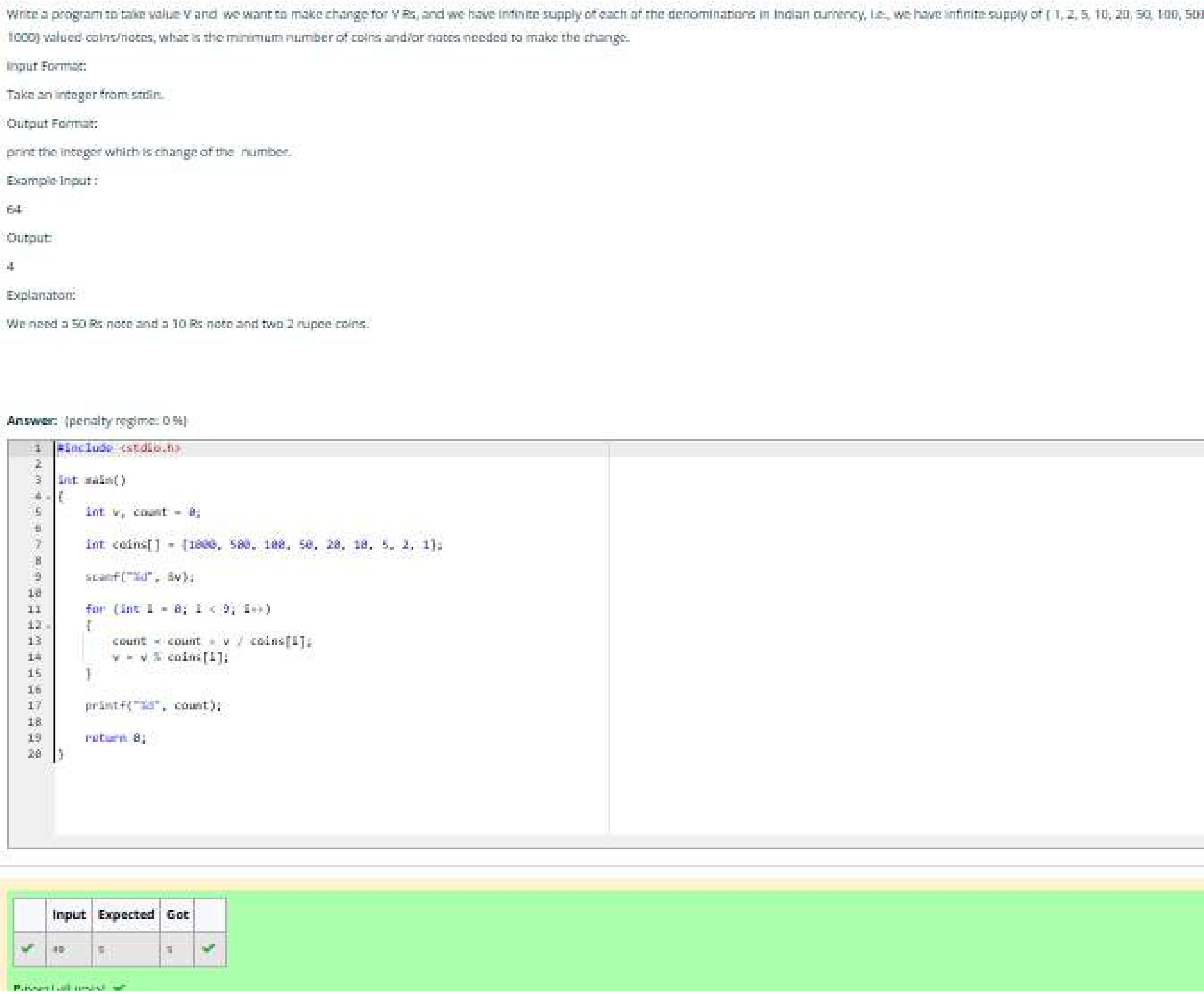
0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

Answer: (penalty regime: 0 %)

```
#include<stdio.h>

int main()
{
    int n,m,i,j,temp,count=0;
    scanf("%d",&n);
    int g[n];
    for(i=0;i<n;i++)
        scanf("%d",&g[i]);
    scanf("%d",&m);
    int s[m];
    for(i=0;i<m;i++)
        scanf("%d",&s[i]);

    for(i=0;i<n;i++)
        for(j=i+1;j<n;j++)
            if(g[i]>g[j])
            {
                temp=g[i];
                g[i]=g[j];
                g[j]=temp;
            }

    for(i=0;i<m;i++)
        for(j=i+1;j<m;j++)
            if(s[i]>s[j])
            {
                temp=s[i];
                s[i]=s[j];
                s[j]=temp;
            }

    i=0;
    j=0;
    while(i<n&&j<m)
    {
        if(s[j]>=g[i])
        {
            count++;
            i++;
        }
        j++;
    }

    printf("%d",count);
    return 0;
}
```

	Input	Expected	Got	
✓	2 1 2 3 1 2 3	2	2	✓



Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500
1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main()
{
    int v, count = 0;

    int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};

    scanf("%d", &v);

    for (int i = 0; i < 9; i++)
    {
        count = count + v / coins[i];
        v = v % coins[i];
    }

    printf("%d", count);

    return 0;
}
```

	Input	Expected	Got	
✓	49	5	5	✓

Passed all tests! ✓